

Sistema Adopcion Equipo7

Especificación de Requerimientos del Sistema

1. Introducción

1.1. Propósito del documento

1.2. Alcance

Alcance general

Alcance en la versión actual

1.3. Definiciones relevantes

2. Actores del sistema

Usuario registrado

Publicador de mascota

Usuario interesado en adopción

Usuario no autenticado o visitante

3. Requerimientos funcionales

RF1: Registro de usuarios en la plataforma

RF2: Publicación de animales en adopción

RF3: Consulta de publicaciones de mascotas disponibles

RF4: Manifestación de interés en una publicación

RF5: Visualización geoespacial segura de mascotas

RF6: Control de acceso al sistema

RF7: Integración con API externa para selección y detalle de raza

RF8: Notificación automatizada y segura por correo electrónico

RF9: Consulta de solicitudes de interés recibidas

RF10: Edición de publicaciones propias de adopción

RF11: Confirmación y finalización de adopción

RF12: Eliminación de publicaciones propias de adopción

4. Casos de uso

CU1: Protección de rutas y registro de usuarios

CU2: Inicio de sesión y consulta de información del usuario

CU3: Actualización de información del usuario

CU4: Despliegue de una publicación de adopción

CU5: **Acceso a publicaciones de adopción**

CU6: Publicación de un animal en adopción

CU7: **Mostrar interés en adoptar una mascota**

CU8: Gestión y edición de publicaciones propias

CU9: Confirmación y finalización de adopción

CU10: Panel de gestión de solicitudes de interés

CU11: Eliminación de publicaciones de adopción

5. Requerimientos no funcionales

	<u>RNF1: Seguridad</u>
	<u>RNF2: Privacidad de la información</u>
	<u>RNF3: Rendimiento</u>
	<u>RNF4: Usabilidad</u>
	<u>RNF5: Disponibilidad</u>
	<u>RNF6: Integridad de datos</u>
6.	<u>Modelado de datos</u>
	<u>Modelo Entidad-Relación</u>
	<u>Entidades principales</u>
	<u>Usuario</u>
	<u>Animales</u>
	<u>Intereses de adopción</u>
7.	<u>Tecnologías utilizadas</u>
	<u>7.1. Backend</u>
	<u>7.2. Frontend</u>
	<u>7.3. Complementos</u>
8.	<u>Arquitectura general</u>
	<u>8.1. Arquitectura Cliente-Servidor</u>
	<u>8.2. Backend en capas</u>
	<u>8.3. Nuevos componentes de Iteración 4</u>
9.	<u>Flujo de autenticación</u>
10.	<u>Endpoints principales</u>
	<u>Usuarios</u>
	<u>Mascotas</u>
	<u>Solicitudes</u>
11.	<u>Limitaciones conocidas</u>
12.	<u>Pruebas recomendadas para validación final</u>
13.	<u>Entregables de la Iteración 4</u>
	<u>Backend</u>
	<u>Frontend</u>
	<u>Documentación</u>

Especificación de Requerimientos del Sistema

1. Introducción

1.1. Propósito del documento

El propósito de este documento es definir de manera precisa y ordenada los requerimientos del Sistema de Adopción de Mascotas, con el fin de establecer una base para su análisis, desarrollo, validación y mantenimiento. Este documento sirve como referencia para el equipo de trabajo, ya que concentra los actores del sistema, los requerimientos funcionales y no funcionales, los casos de uso, la arquitectura general, el flujo de autenticación y el modelo conceptual de datos.

El historial de cambios entre cada versión del proyecto se muestra a continuación.

Versión	Cambios
✓ 1.0	Versión inicial. Funcionalidades básicas de usuario.
✓ 2.0	Integración del backend con el frontend. Autenticación con token. Nueva funcionalidad de actualización de datos para usuario.
✓ 3.0	Expansión funcional del sistema mediante publicación, consulta, detalle e interés en mascotas.
➡ 4.0	Consolidación final del sistema. Se agregan edición de publicaciones propias, confirmación de adopción, panel de solicitudes de interés y eliminación de publicaciones con confirmación.

1.2. Alcance

Alcance general

El sistema completo consiste en una plataforma orientada a la publicación y consulta de mascotas en adopción. A través de esta plataforma, los usuarios registrados pueden crear publicaciones de mascotas disponibles para adopción, consultar publicaciones existentes, manifestar interés por una mascota y gestionar el ciclo de vida de sus propias publicaciones.

El sistema busca facilitar el proceso de adopción de manera segura, evitando la exposición pública e indiscriminada de información sensible de los usuarios. Los datos de contacto de los interesados se muestran únicamente al usuario propietario de la publicación correspondiente, con el propósito de facilitar la comunicación inicial entre publicador e interesado.

Alcance en la versión actual

En esta cuarta versión, el sistema consolida el flujo principal de adopción. Además de las funcionalidades de usuario y del catálogo de mascotas ya

integradas en versiones anteriores, se incorporan funcionalidades para que el publicador administre sus publicaciones, revise solicitudes de interés, confirme la adopción de una mascota y elimine publicaciones propias cuando sea necesario.

A través de las interfaces y endpoints actuales, el sistema permite:

- **Funcionalidades básicas de usuario:** registro, inicio y cierre de sesión, consulta y actualización de perfil.
- **Publicación de mascotas:** los usuarios autenticados pueden registrar animales en el sistema mediante formularios.
- **Exploración de catálogo:** acceso a una vista general con las mascotas disponibles.
- **Consulta a detalle:** visualización individual de la información completa de una mascota.
- **Manifestación de interés:** los usuarios autenticados pueden marcar interés por una mascota disponible.
- **Gestión de publicaciones propias:** el dueño de una publicación puede editar datos de la mascota mientras la publicación se encuentre disponible.
- **Panel de solicitudes:** el publicador puede consultar las personas interesadas en sus publicaciones.
- **Confirmación de adopción:** el publicador puede cerrar el proceso marcando una mascota como adoptada.
- **Eliminación de publicaciones:** el dueño puede eliminar una publicación disponible, previa confirmación mediante Pop-In.

Todas estas funciones se comunican con el backend real mediante peticiones HTTP, respetando la autenticación por token en las rutas protegidas.

1.3. Definiciones relevantes

- **ERS:** Especificación de Requerimientos del Sistema. Documento que describe funciones, restricciones y comportamiento esperado del sistema.
- **RF:** Requerimiento Funcional. Define una función o servicio que el sistema debe proporcionar.
- **RNF:** Requerimiento No Funcional. Define atributos de calidad o restricciones del sistema, como seguridad, disponibilidad, rendimiento o

usabilidad.

- **CU:** Caso de Uso. Describe la interacción entre un actor y el sistema para alcanzar un objetivo específico.
- **Token:** Identificador generado por el backend durante el inicio de sesión y usado por el frontend en el encabezado `Authorization` para acceder a funciones protegidas.
- **Pop-In:** Ventana emergente o modal dentro de la interfaz que solicita confirmación antes de ejecutar una acción sensible, como eliminar una publicación.

2. Actores del sistema

Usuario registrado

Persona que ha creado una cuenta dentro de la plataforma y puede interactuar con las funcionalidades privadas del sistema. Este actor puede consultar publicaciones, marcar interés en mascotas, publicar animales en adopción y administrar sus propias publicaciones.

Publicador de mascota

Usuario registrado que crea una publicación de una mascota en adopción. Puede consultar solicitudes de interés asociadas a sus mascotas, editar publicaciones propias, confirmar adopciones y eliminar publicaciones disponibles.

Usuario interesado en adopción

Usuario registrado que consulta publicaciones disponibles y utiliza la opción "Me interesa" para manifestar su intención de adoptar una mascota.

Usuario no autenticado o visitante

Persona que accede a la plataforma sin haber iniciado sesión. Su interacción está limitada a las vistas públicas de registro e inicio de sesión. Si intenta acceder a vistas protegidas, el sistema lo redirige a la pantalla de autenticación.

3. Requerimientos funcionales

RF1: Registro de usuarios en la plataforma

Descripción:

Cualquier persona interesada en interactuar con la plataforma debe registrarse proporcionando nombre, correo electrónico, código postal y contraseña.

Criterios de aceptación:

- El sistema debe validar que los campos obligatorios no estén vacíos.
- El correo electrónico no debe estar registrado previamente.
- La contraseña debe almacenarse usando un mecanismo de hash y no en texto plano.
- Al completar el registro exitosamente, el usuario debe ser redirigido a la pantalla de inicio de sesión.

RF2: Publicación de animales en adopción

Descripción:

Un usuario autenticado puede publicar una mascota en adopción proporcionando nombre, descripción, fotografía, tipo, raza y código postal.

Criterios de aceptación:

- El usuario debe contar con sesión activa.
- El sistema debe validar que los campos obligatorios estén completos.
- La imagen debe enviarse como `fotoBase64`.
- La publicación debe guardarse con estado `DISPONIBLE`.

RF3: Consulta de publicaciones de mascotas disponibles

Descripción:

El sistema permite consultar las publicaciones de mascotas disponibles para adopción, mostrando datos generales como nombre, tipo, raza, fotografía y código postal.

Criterios de aceptación:

- El catálogo general debe mostrar únicamente mascotas en estado `DISPONIBLE`.
- El usuario puede buscar publicaciones por nombre.

- El usuario puede acceder al detalle de una publicación.

RF4: Manifestación de interés en una publicación

Descripción:

El sistema permite que un usuario autenticado manifieste interés en adoptar una mascota mediante la opción "Me interesa".

Criterios de aceptación:

- El usuario debe contar con sesión activa.
- El sistema debe registrar el interés asociado al usuario y a la mascota.
- El sistema debe impedir registros duplicados de interés sobre la misma mascota por el mismo usuario.
- La publicación debe encontrarse disponible.

RF5: Visualización geoespacial segura de mascotas

Descripción:

El sistema contempla como funcionalidad futura una visualización geoespacial segura basada en código postal, sin exponer direcciones exactas.

Estado actual:

Esta funcionalidad queda identificada como alcance futuro o pendiente de implementación.

RF6: Control de acceso al sistema

Descripción:

El sistema restringe el acceso a vistas y funciones protegidas mediante autenticación por token.

Criterios de aceptación:

- El frontend debe validar la existencia de token en `sessionStorage` para acceder a vistas privadas.
- El backend debe validar el token recibido en el encabezado `Authorization`.
- Si el token no existe o no es válido, el sistema debe rechazar la petición.

RF7: Integración con API externa para selección y detalle de raza

Descripción:

Se contempla la integración futura con APIs externas de razas de perros y gatos para enriquecer la información de las publicaciones.

Estado actual:

Esta funcionalidad queda como alcance futuro o pendiente de implementación.

RF8: Notificación automatizada y segura por correo electrónico**Descripción:**

Se contempla el envío automatizado de notificaciones por correo electrónico cuando un usuario manifieste interés en una mascota.

Estado actual:

En la versión actual, la comunicación inicial se apoya en el panel de solicitudes de interés, donde el publicador puede consultar los datos de contacto del interesado. El envío automático por correo queda como alcance futuro.

RF9: Consulta de solicitudes de interés recibidas**Descripción:**

El sistema permite que el usuario publicador consulte las solicitudes de interés registradas sobre sus propias publicaciones de adopción.

Criterios de aceptación:

- El usuario debe contar con sesión activa.
- El sistema solo debe mostrar solicitudes asociadas a publicaciones del usuario autenticado.
- Cada publicación debe mostrar sus interesados, incluyendo nombre, correo electrónico y fecha de interés.
- Si el usuario no tiene publicaciones o no tiene solicitudes, la interfaz debe mostrar un mensaje informativo.
- Los datos de contacto de los interesados solo deben mostrarse al dueño de la publicación.

RF10: Edición de publicaciones propias de adopción**Descripción:**

El sistema permite que el usuario propietario de una publicación edite los datos de una mascota publicada mientras se encuentre en estado **DISPONIBLE**.

Criterios de aceptación:

- El usuario debe contar con sesión activa.
- El sistema debe verificar que la publicación exista.
- El sistema debe verificar que el usuario autenticado sea el dueño de la publicación.
- Solo deben editarse publicaciones en estado **DISPONIBLE**.
- El sistema debe persistir los cambios en la base de datos.
- El sistema debe retornar la publicación actualizada.

RF11: Confirmación y finalización de adopción

Descripción:

El sistema permite que el dueño de una publicación marque una mascota como adoptada, cerrando su disponibilidad dentro del catálogo.

Criterios de aceptación:

- El usuario debe contar con sesión activa.
- La publicación debe existir y pertenecer al usuario autenticado.
- La publicación debe encontrarse en estado **DISPONIBLE**.
- Al confirmar la adopción, el estado de la mascota debe cambiar a **ADOPTADO**.
- Una mascota adoptada debe dejar de aparecer en el catálogo general de mascotas disponibles.
- El sistema debe impedir confirmar nuevamente una mascota ya adoptada.

RF12: Eliminación de publicaciones propias de adopción

Descripción:

El sistema permite que el usuario propietario de una publicación elimine una publicación propia que todavía se encuentre disponible, previa confirmación explícita desde la interfaz.

Criterios de aceptación:

- El usuario debe contar con sesión activa.

- El sistema debe verificar que la publicación exista.
- El sistema debe verificar que el usuario autenticado sea dueño de la publicación.
- La publicación debe encontrarse en estado `DISPONIBLE`.
- La interfaz debe solicitar confirmación mediante Pop-In antes de ejecutar la eliminación.
- Si el usuario cancela la confirmación, no se debe enviar la petición al backend.
- Al eliminarse, la publicación debe dejar de aparecer en las vistas del sistema.

4. Casos de uso

CU1: Protección de rutas y registro de usuarios

Actores: Usuario no autenticado.

Precondiciones: El usuario no cuenta con sesión activa.

Flujo principal:

1. El usuario intenta acceder a una vista protegida.
2. El frontend verifica si existe token en `sessionStorage`.
3. Si no existe token, el sistema redirige al usuario a `login.html`.
4. El usuario accede a `register.html`.
5. El usuario llena el formulario de registro.
6. El frontend valida campos obligatorios.
7. El frontend envía una petición `POST /usuarios/register`.
8. El backend valida que el correo no esté registrado.
9. El backend guarda el usuario con contraseña hasheada.
10. El frontend muestra mensaje de éxito y redirige a `login.html`.

Escenarios de error:

- Campos obligatorios vacíos.
- Correo ya registrado.

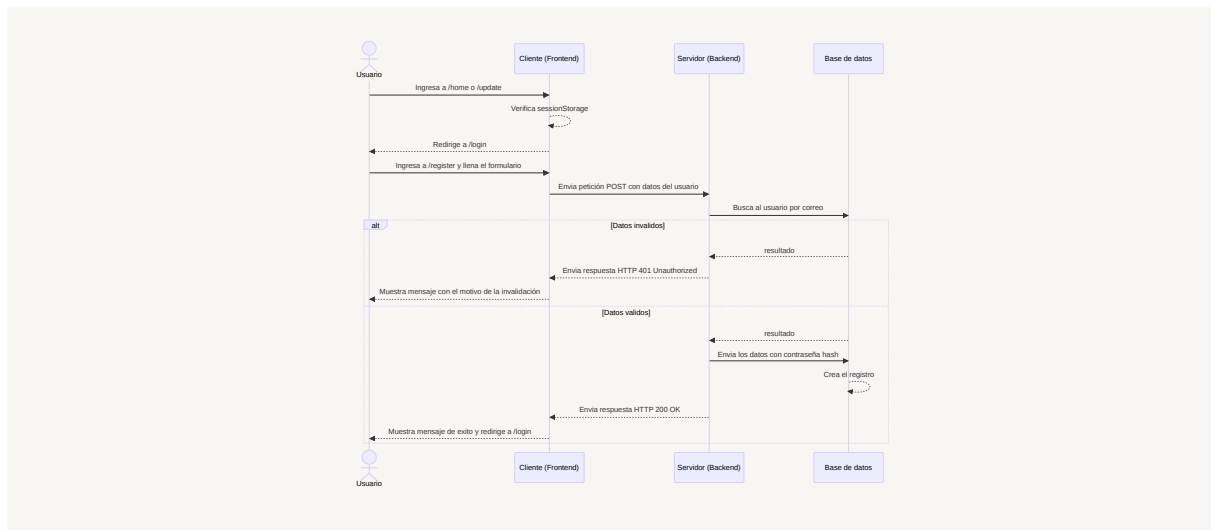
- Error de comunicación con el servidor.

Postcondición: El usuario queda registrado en el sistema.

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos

    Usuario->>UI : Ingresa a /home o /update
    UI-->>UI : Verifica sessionStorage
    UI-->>Usuario : Redirige a /login
    Usuario->>UI : Ingresa a /register y llena el formulari
o
    UI->>API : Envía petición POST con datos del usuario
    API->>DB : Busca al usuario por correo
    alt Datos invalidos
        DB-->>API : resultado
        API-->>UI : Envía respuesta HTTP 401 Unauthorized
        UI-->>Usuario : Muestra mensaje con el motivo de la
invalidación
    else Datos validos
        DB-->>API : resultado
        API->>DB : Envía los datos con contraseña hash
        DB-->>DB : Crea el registro
        API-->>UI : Envía respuesta HTTP 200 OK
        UI-->>Usuario : Muestra mensaje de éxito y redirige
a /login
    end
end
```



CU2: Inicio de sesión y consulta de información del usuario

Actores: Usuario registrado.

Precondiciones: El usuario cuenta con un registro válido.

Flujo principal:

1. El usuario ingresa correo y contraseña en `login.html`.
2. El frontend envía `POST /usuarios/login`.
3. El backend valida credenciales.
4. Si son correctas, genera un token y lo asocia al usuario.
5. El frontend guarda `token`, `email` y `userId` en `sessionStorage`.
6. El usuario es redirigido a `home.html`.
7. El frontend consulta `GET /usuarios/me` con el token en `Authorization`.
8. El backend retorna los datos del usuario.
9. El frontend despliega la información del perfil.

Escenarios de error:

- Credenciales inválidas.
- Token inexistente o inválido.
- Error de comunicación.

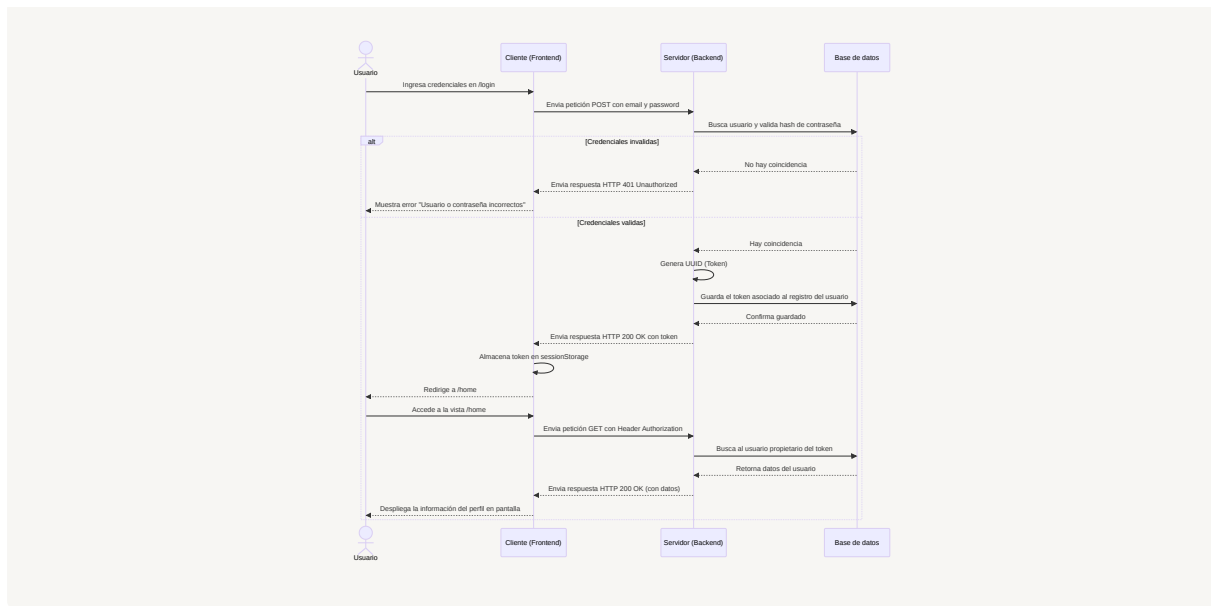
Postcondición: El usuario tiene una sesión activa y visualiza su perfil.

Diagrama de secuencia:

```

sequenceDiagram
actor Usuario as Usuario
participant UI as Cliente (Frontend)
participant API as Servidor (Backend)
participant DB as Base de datos
Usuario->>UI : Ingresa credenciales en /login
UI->>API : Envía petición POST con email y password
API->>DB : Busca usuario y valida hash de contraseña
alt Credenciales invalidas
DB-->>API : No hay coincidencia
API-->>UI : Envía respuesta HTTP 401 Unauthorized
UI-->>Usuario : Muestra error "Usuario o contraseña incorre
ctos"
else Credenciales validas
DB-->>API : Hay coincidencia
API->>API : Genera UUID (Token)
API->>DB : Guarda el token asociado al registro del usuario
DB-->>API : Confirma guardado
API-->>UI : Envía respuesta HTTP 200 OK con token
UI->>UI : Almacena token en sessionStorage
UI-->>Usuario : Redirige a /home
Usuario->>UI : Accede a la vista /home
UI->>API : Envía petición GET con Header Authorization
API->>DB : Busca al usuario propietario del token
DB-->>API : Retorna datos del usuario
API-->>UI : Envía respuesta HTTP 200 OK (con datos)
UI-->>Usuario : Despliega la información del perfil en pant
alla
end

```



CU3: Actualización de información del usuario

Actores: Usuario autenticado.

Precondiciones: El usuario tiene una sesión activa.

Flujo principal:

1. El usuario accede a `update.html`.
2. El frontend carga los datos actuales con `GET /usuarios/me`.
3. El usuario modifica nombre, correo, código postal o contraseña.
4. El frontend valida campos obligatorios.
5. El frontend envía `PUT /usuarios` con el token.
6. El backend identifica al usuario por token.
7. El backend actualiza los datos.
8. El frontend muestra mensaje de éxito y redirige a `home.html`.

Escenarios de error:

- Campos obligatorios vacíos.
- Correo ocupado por otro usuario.
- Token inválido.

Postcondición: La información del usuario queda actualizada.

Diagrama de secuencia:

sequenceDiagram

actor Usuario as Usuario

participant UI as Cliente (Frontend)

participant API as Servidor (Backend)

participant DB as Base de datos

Usuario->>UI : Modifica datos en /update y da clic en Guardar

UI->>API : Envía petición PUT con datos y Header Authorization

API->>DB : Valida al usuario por token y busca el correo
alt Correo ocupado

DB-->>API : Resultado de conflicto

API-->>UI : Envía respuesta HTTP 409 Correo ocupado

UI-->>Usuario : Muestra mensaje de error

else Correo disponible

DB-->>API : Resultado exitoso

API->>API : Hashea la nueva contraseña (si se envió una)

API->>DB : Sobreescribe los atributos en la base de datos
os

DB-->>API : Confirma la actualización

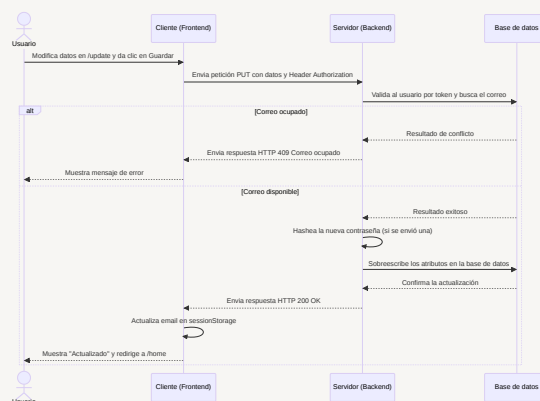
API-->>UI : Envía respuesta HTTP 200 OK

UI->>UI : Actualiza email en sessionStorage

UI-->>Usuario : Muestra "Actualizado" y redirige a /home

e

end



CU4: Despliegue de una publicación de adopción

Actores: Usuario registrado.

Precondiciones: Existe una publicación registrada en el sistema.

Flujo principal:

1. El usuario selecciona una mascota desde el catálogo.
2. El frontend redirige a `publication_view.html?id={id}`.
3. El frontend obtiene el identificador desde la URL.
4. El frontend envía `GET /mascotas/detalle/{id}`.
5. El backend busca la publicación.
6. Si existe, retorna sus datos completos.
7. El frontend muestra nombre, foto, descripción, tipo, raza, código postal, usuario dueño y estado.

Flujos alternativos:

- El usuario accede al detalle desde búsqueda.
- El usuario accede al detalle desde "Mis publicaciones".
- El usuario accede al detalle desde "Me interesa".

Escenarios de error:

- Publicación no encontrada.
- Error de comunicación con el backend.

Postcondición: El usuario visualiza la información detallada de una publicación.

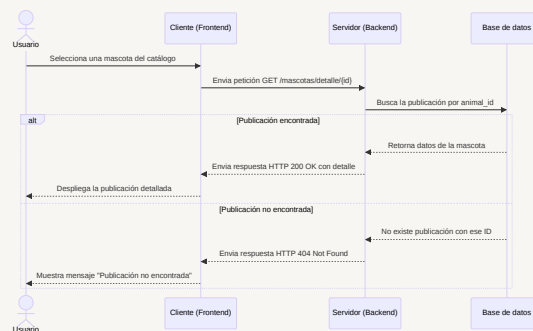
Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos
    Usuario->>UI : Selecciona una mascota del catálogo
    UI->>API : Envía petición GET /mascotas/detalle/{id}
    API->>DB : Busca la publicación por animal_id
```

```

alt Publicación encontrada
  DB-->>API : Retorna datos de la mascota
  API-->>UI : Envía respuesta HTTP 200 OK con detalle
  UI-->>Usuario : Despliega la publicación detallada
else Publicación no encontrada
  DB-->>API : No existe publicación con ese ID
  API-->>UI : Envía respuesta HTTP 404 Not Found
  UI-->>Usuario : Muestra mensaje "Publicación no encontrada"
end

```



CU5: Acceso a publicaciones de adopción

Actores: Usuario registrado.

Actores secundarios: Publicador de mascota, interesado en adopción.

Precondiciones:

- El usuario cuenta con una sesión activa dentro de la plataforma.
- Existen publicaciones de mascotas registradas en el sistema.
- Para consultar las pestañas "Mis publicaciones" o "Me interesa", el usuario debe contar con un token válido.

Flujo Principal (Escenario normal):

1. El usuario ingresa a la vista de exploración o catálogo de mascotas disponibles.
2. El cliente solicita al servidor el listado de publicaciones mediante el endpoint `GET /mascotas.`
3. El servidor consulta en la base de datos las publicaciones de mascotas registradas.

4. El servidor recupera la información general de cada publicación, como nombre, fotografía, tipo, raza, descripción, código postal y estado de publicación.
5. El servidor responde al cliente con una lista de publicaciones.
6. El cliente recibe la lista y despliega las publicaciones en una vista de tarjetas o lista deslizable.
7. El usuario puede desplazarse por la pantalla para revisar las mascotas disponibles.
8. Si el usuario selecciona una publicación específica, el sistema puede continuar con el caso de uso CU3 para mostrar el detalle de la mascota.

Flujos Alternativos:

Escenarios alternos:

- Búsqueda por nombre:
El usuario escribe el nombre de una mascota en la barra de búsqueda. El cliente envía una petición al servidor usando el parámetro `nombre`, por ejemplo `GET /mascotas?nombre=luna`. El servidor filtra las publicaciones cuyo nombre coincida parcial o totalmente con el texto ingresado y devuelve los resultados encontrados.
- Consulta de "Mis publicaciones":
El usuario selecciona la pestaña "Mis publicaciones". El cliente envía una petición al servidor usando el parámetro `filtro=mis-publicaciones` y agrega el token en el header `Authorization`. El servidor identifica al usuario autenticado mediante el token y devuelve únicamente las publicaciones creadas por ese usuario.
- Consulta de "Me interesa":
El usuario selecciona la pestaña "Me interesa". El cliente envía una petición al servidor usando el parámetro `filtro=me-interesa` y agrega el token en el header `Authorization`. El servidor identifica al usuario autenticado y consulta las publicaciones que ese usuario marcó previamente como interés.
- Combinación de búsqueda y filtro:
El usuario puede buscar por nombre dentro de una pestaña específica. Por ejemplo, puede buscar "Luna" dentro de la pestaña "Me interesa" mediante `GET /mascotas?filtro=me-interesa&nombre=luna`. El servidor aplica ambos criterios y devuelve únicamente las publicaciones correspondientes.

Escenarios de error:

- Si no existen publicaciones registradas en la base de datos, el sistema muestra una lista vacía o un mensaje indicando que no hay mascotas disponibles.
- Si el usuario busca un nombre que no coincide con ninguna publicación, el sistema muestra un mensaje indicando que no se encontraron resultados.
- Si el usuario intenta acceder a "Mis publicaciones" o "Me interesa" sin enviar token, el servidor responde con 401 Unauthorized y el cliente solicita iniciar sesión.
- Si el token enviado no corresponde a una sesión válida, el servidor rechaza la petición y el cliente redirige al usuario a la pantalla de inicio de sesión.
- Si ocurre un error de comunicación entre cliente y servidor, el sistema muestra un mensaje de error y evita mostrar datos simulados o incompletos.

Postcondición:

El usuario visualiza las publicaciones disponibles o filtradas según su búsqueda. No se modifica la información de las publicaciones, ya que este caso de uso es de consulta. Si el usuario selecciona una publicación específica, el flujo continúa con el caso de uso CU3.

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos
    Usuario->>UI : Ingresa al catálogo de mascotas
    UI->>API : Envía petición GET /mascotas con filtros opcionales
    API->>DB : Consulta publicaciones según nombre y/o filtro

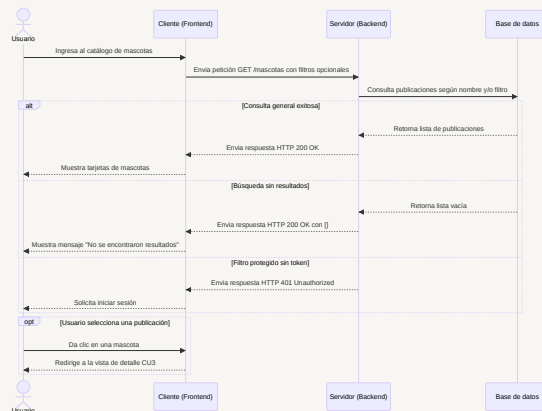
    alt Consulta general exitosa
        DB-->>API : Retorna lista de publicaciones
        API-->>UI : Envía respuesta HTTP 200 OK
        UI-->>Usuario : Muestra tarjetas de mascotas
    end
```

```

else Búsqueda sin resultados
    DB-->>API : Retorna lista vacía
    API-->>UI : Envía respuesta HTTP 200 OK con []
    UI-->>Usuario : Muestra mensaje "No se encontraron resultados"
else Filtro protegido sin token
    API-->>UI : Envía respuesta HTTP 401 Unauthorized
    UI-->>Usuario : Solicita iniciar sesión
end

opt Usuario selecciona una publicación
    Usuario-->>UI : Da clic en una mascota
    UI-->>Usuario : Redirige a la vista de detalle CU3
end

```



CU6: Publicación de un animal en adopción

Actores: Usuario autenticado

Precondiciones:

- El usuario cuenta con una sesión activa dentro de la plataforma.

Flujo Principal (Escenario normal)

- El usuario accede a la vista protegida de publicación de mascotas (`/publish_mascot`).
- El usuario llena el formulario con los datos requeridos del animal (nombre, descripción, fotografía, tipo, raza y código postal).
- El usuario da clic en el botón de "Publicar".

4. El sistema cliente valida internamente que ninguno de los campos obligatorios se encuentre vacío.
5. El cliente envía una petición al servidor con los datos de la mascota. El servidor recibe la petición y almacena en la base de datos, asignándole automáticamente el estado `DISPONIBLE` a la publicación.
6. El servidor responde con un estado de éxito. El cliente recibe la confirmación, muestra el mensaje en pantalla "¡Mascota publicada con éxito!" y redirige automáticamente al usuario a la vista del catálogo (`mascotas.html`) para que visualice su nueva publicación.

Flujos alternativos:

Escenarios alternos:

- El usuario, sin haber iniciado sesión previamente, intenta acceder directamente a la URL de publicación copiando y pegando el enlace en su navegador. El sistema detecta la ausencia del token, bloquea el acceso y redirige de inmediato al usuario a `login.html`.

Escenarios de error:

- Campos incompletos: El usuario omite ingresar información crítica (como la descripción o la fotografía) y presiona el botón de publicar. El sistema cliente detiene la ejecución de la publicación, y muestra una advertencia visual solicitando rellenar la información faltante.

Postcondición:

La mascota cuenta con su propia publicación con toda la información relevante y en estado disponible, para que más usuarios puedan verla en la página de Explorar Mascotas.

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario autenticado
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos

    Usuario->>UI: Accede a publish_mascot.html
    UI-->>UI: Verifica token en sessionStorage
```

```

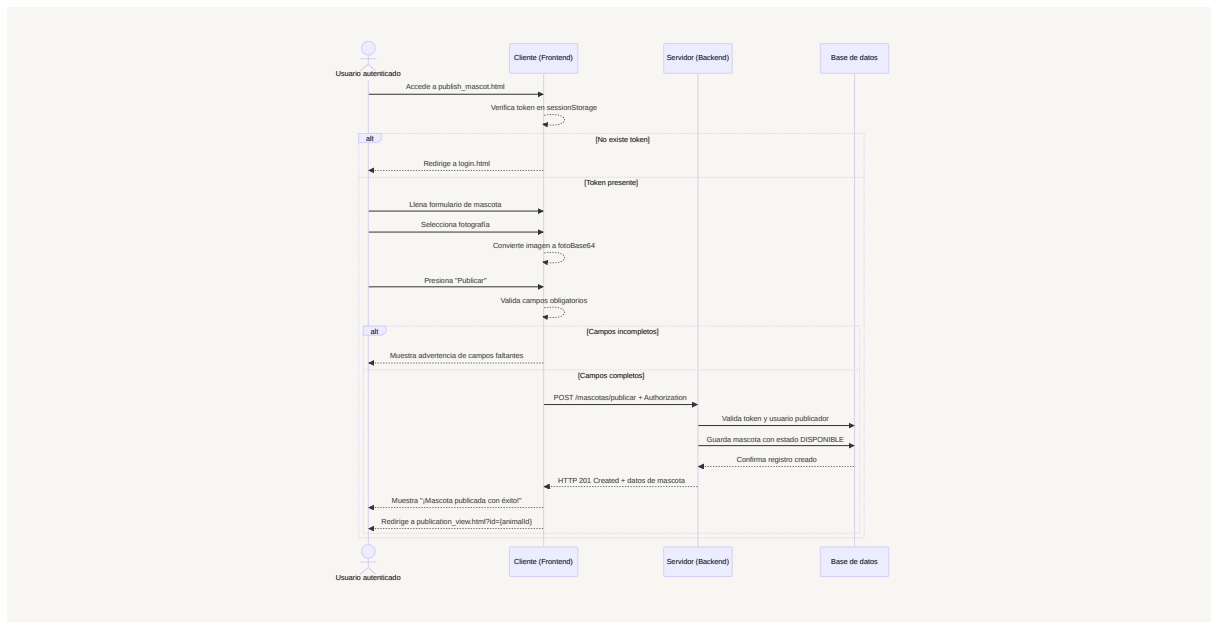
alt No existe token
    UI-->>Usuario: Redirige a login.html
else Token presente
    Usuario->>UI: Llena formulario de mascota
    Usuario->>UI: Selecciona fotografía
    UI-->>UI: Convierte imagen a fotoBase64
    Usuario->>UI: Presiona "Publicar"
    UI-->>UI: Valida campos obligatorios

    alt Campos incompletos
        UI-->>Usuario: Muestra advertencia de campos faltantes
    else Campos completos
        UI->>API: POST /mascotas/publicar + Authorization

        API->>DB: Valida token y usuario publicador
        API->>DB: Guarda mascota con estado DISPONIBLE
        DB-->>API: Confirma registro creado
        API-->>UI: HTTP 201 Created + datos de mascota
        UI-->>Usuario: Muestra "¡Mascota publicada con éxito!"

        UI-->>Usuario: Redirige a publication_view.html?id={animalId}
    end
end
end

```



CU7: Mostrar interés en adoptar una mascota

Actores: Usuario autenticado

Precondiciones:

- El usuario cuenta con una sesión activa dentro de la plataforma.
- Existe al menos una publicación de una mascota cuyo `estado_publicacion` es `DISPONIBLE`.

Flujo Principal (Escenario normal)

1. El usuario accede a la vista de exploración de mascotas (`mascotas.html`). Para registrar interés, el sistema requiere sesión activa.
2. El usuario, tras revisar la información y fotografía del animal, presiona el botón interactivo de "Me interesa".
3. El sistema cliente captura el evento y realiza una petición al servidor. El servidor recibe la solicitud, verifica el token, valida la existencia del animal solicitado mediante su ID, y finalmente guarda un nuevo registro de interés en la base de datos.
4. El servidor confirma la transacción devolviendo un HTTP 200 OK. El cliente notifica al usuario del éxito de la operación mostrando un mensaje de confirmación.

Flujos alternativos:

Escenarios alternos:

- El usuario, sin haber iniciado sesión previamente, intenta acceder directamente a la URL de publicación copiando y pegando el enlace en su navegador. El sistema detecta la ausencia del token, bloquea el acceso y redirige de inmediato al usuario a `login.html`.

Escenarios de error:

- El usuario presiona el botón de interés sobre una mascota por la cual ya había generado una solicitud previa. El servidor intenta realizar el registro pero la base de datos detecta una violación, rechazando la transacción y devuelve un error. El cliente captura la respuesta y alerta al usuario: "Ya registraste interés en esta mascota".

Postcondición:

El usuario ha dejado sentado un interés formal de una mascota en la base de datos. Potencialmente, este sería el primer paso para el proceso de adopción formal.

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario autenticado
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos

    Usuario->>UI: Accede a mascotas.html
    UI-->>UI: Verifica si existe token

    alt No existe token
        UI-->>Usuario: Redirige a login.html
    else Token presente
        UI->>API: GET /mascotas
        API->>DB: Consulta publicaciones DISPONIBLES
        DB-->>API: Retorna lista de mascotas
        API-->>UI: HTTP 200 OK + publicaciones
        UI-->>Usuario: Muestra tarjetas de mascotas

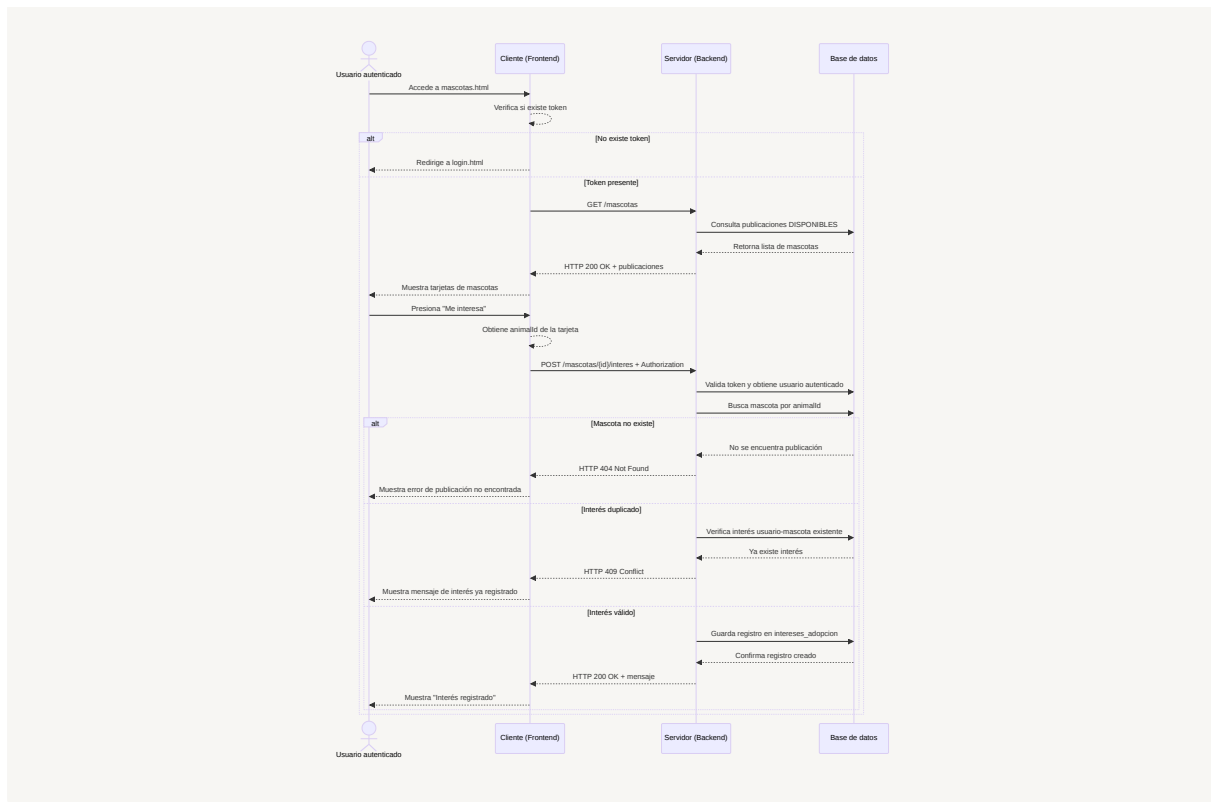
        Usuario->>UI: Presiona "Me interesa"
```

```

    UI-->>UI: Obtiene animalId de la tarjeta
    UI-->>API: POST /mascotas/{id}/interes + Authorizati
on
    API-->>DB: Valida token y obtiene usuario autenticad
o
    API-->>DB: Busca mascota por animalId

    alt Mascota no existe
        DB-->>API: No se encuentra publicación
        API-->>UI: HTTP 404 Not Found
        UI-->>Usuario: Muestra error de publicación no
encontrada
    else Interés duplicado
        API-->>DB: Verifica interés usuario-mascota exis
tente
        DB-->>API: Ya existe interés
        API-->>UI: HTTP 409 Conflict
        UI-->>Usuario: Muestra mensaje de interés ya re
gistrado
    else Interés válido
        API-->>DB: Guarda registro en intereses_adopcion
        DB-->>API: Confirma registro creado
        API-->>UI: HTTP 200 OK + mensaje
        UI-->>Usuario: Muestra "Interés registrado"
    end
end
end

```



CU8: Gestión y edición de publicaciones propias

Actores: Usuario autenticado dueño de la publicación.

Precondiciones:

- El usuario cuenta con un token válido.
- La publicación existe.
- El usuario autenticado es el dueño original de la publicación.
- La publicación se encuentra en estado **DISPONIBLE**.

Flujo principal:

1. El dueño accede a la vista de edición de una publicación propia.
2. El frontend obtiene el identificador de la mascota desde la URL.
3. El frontend carga los datos actuales mediante `GET /mascotas/detalle/{id}`.
4. El sistema llena el formulario con los datos actuales.
5. El usuario modifica la información de la mascota.
6. El usuario presiona "Guardar".
7. El frontend valida que los campos obligatorios estén completos.

8. El frontend envía una petición `PUT /mascotas/editar/{id}` con el token en `Authorization`.
9. El backend valida el token.
10. El backend verifica que la publicación exista.
11. El backend verifica que el usuario autenticado sea el dueño.
12. El backend verifica que la publicación esté `DISPONIBLE`.
13. El backend actualiza los datos en la tabla de animales.
14. El backend retorna la mascota actualizada.
15. El frontend muestra un mensaje de éxito y redirige a la vista de detalle.

Escenarios alternos:

- El usuario cambia la fotografía de la mascota; el frontend convierte la nueva imagen a `fotoBase64` antes de enviarla.
- El usuario solo modifica algunos campos, manteniendo la información restante.

Escenarios de error:

- **401 Unauthorized:** token faltante o inválido.
- **403 Forbidden:** el usuario no es dueño de la publicación.
- **404 Not Found:** la publicación no existe.
- **409 Conflict:** la publicación ya no está disponible o está adoptada.
- **500 Internal Server Error:** error inesperado del servidor.

Postcondición: La publicación queda actualizada en la base de datos y mantiene su estado disponible.

Endpoint utilizado:

`PUT /mascotas/editar/{id}`

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario dueño
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos
```

```

Usuario->>UI: Accede a update_mascot.html?id={id}
UI-->>UI: Verifica token en sessionStorage
UI-->>UI: Obtiene animalId desde la URL

alt No existe token
    UI-->>Usuario: Redirige a login.html
else Token presente
    UI->>API: GET /mascotas/detalle/{id}
    API->>DB: Busca publicación por animalId

    alt Publicación no encontrada
        DB-->>API: No existe publicación
        API-->>UI: HTTP 404 Not Found
        UI-->>Usuario: Muestra error de publicación no
encontrada
    else Publicación encontrada
        DB-->>API: Retorna datos de mascota
        API-->>UI: HTTP 200 OK + datos de mascota
        UI-->>Usuario: Llena formulario con datos actua
les

    alt Publicación no disponible
        UI-->>Usuario: Bloquea edición y muestra qu
e no puede modificarse
    else Publicación disponible
        Usuario->>UI: Modifica datos de la mascota
        opt Cambia fotografía
            Usuario->>UI: Selecciona nueva imagen
            UI-->>UI: Convierte imagen a fotoBase64
        end

        Usuario->>UI: Presiona "Guardar"
        UI-->>UI: Valida campos obligatorios

    alt Campos incompletos
        UI-->>Usuario: Muestra advertencia de c
ampos faltantes

```

```

else Campos completos
    UI->>API: PUT /mascotas/editar/{id} + A
    uthorization
        API->>DB: Valida token y obtiene usuari
    o autenticado
        API->>DB: Busca publicación por animalI
    d

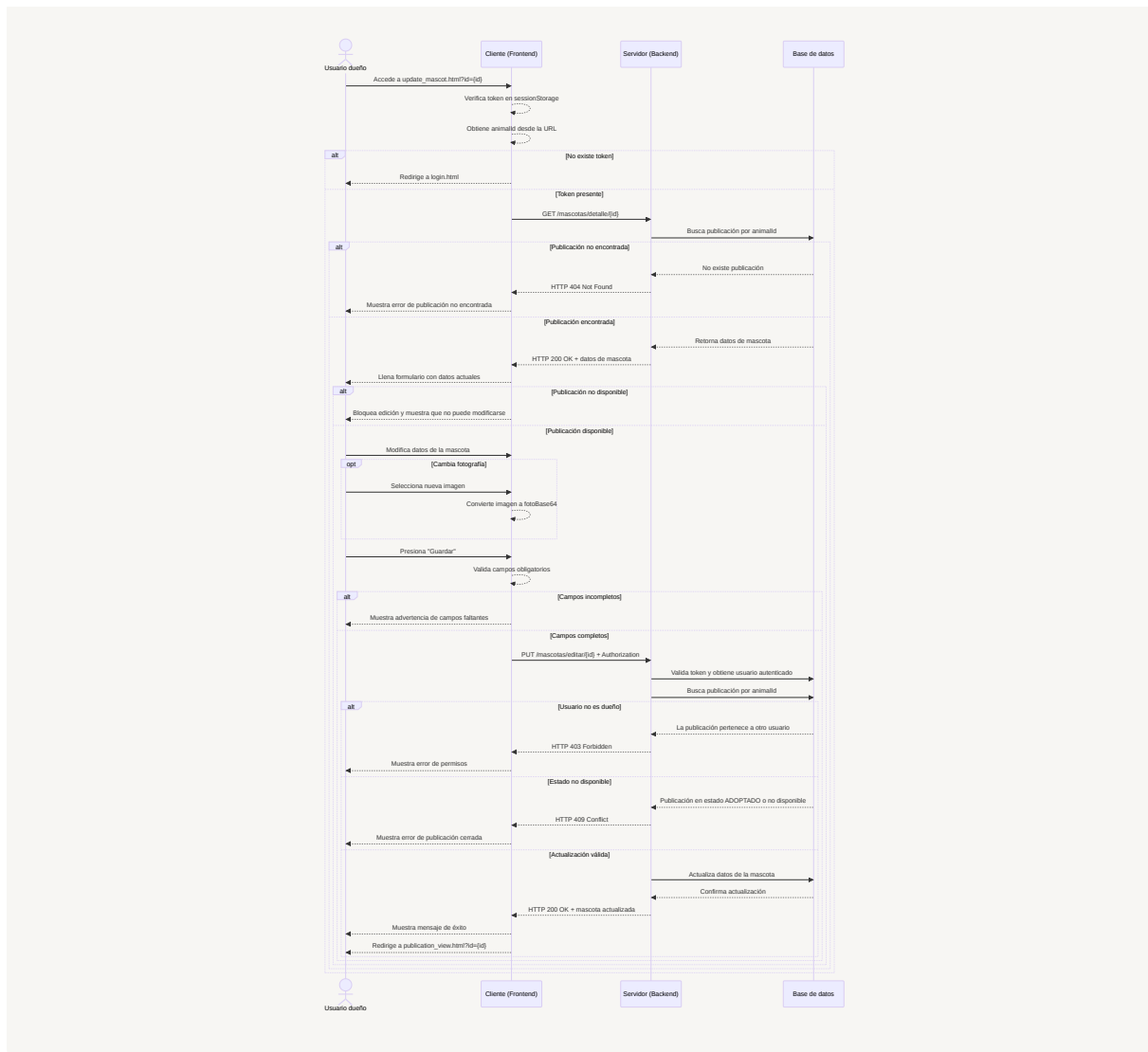
    alt Usuario no es dueño
        DB-->>API: La publicación pertenece
    a otro usuario
        API-->>UI: HTTP 403 Forbidden
        UI-->>Usuario: Muestra error de per
    misos

    else Estado no disponible
        DB-->>API: Publicación en estado AD
    OPTADO o no disponible
        API-->>UI: HTTP 409 Conflict
        UI-->>Usuario: Muestra error de pub
    licación cerrada

    else Actualización válida
        API->>DB: Actualiza datos de la mas
    cota
        DB-->>API: Confirma actualización
        API-->>UI: HTTP 200 OK + mascota ac
    tualizada
        UI-->>Usuario: Muestra mensaje de é
    xito

        UI-->>Usuario: Redirige a publicati
    on_view.html?id={id}
    end
    end
    end
    end
    end
end

```



CU9: Confirmación y finalización de adopción

Actores: Usuario autenticado dueño de la publicación.

Precondiciones:

- El usuario cuenta con una sesión activa.
- La publicación existe.
- La publicación pertenece al usuario autenticado.
- La mascota se encuentra en estado **DISPONIBLE**.

Flujo principal:

1. El publicador accede al detalle de una mascota propia.

2. El sistema identifica que el usuario autenticado es dueño de la publicación.
3. El frontend muestra la opción "Confirmar adopción".
4. El usuario presiona "Confirmar adopción".
5. El frontend envía una petición `PUT /mascotas/{id}/adoptar` con el token en `Authorization`.
6. El backend valida el token.
7. El backend verifica que la publicación exista.
8. El backend verifica que el usuario autenticado sea el dueño.
9. El backend verifica que la mascota esté `DISPONIBLE`.
10. El backend actualiza `estado_publicacion` a `ADOPTADO`.
11. El backend responde con éxito.
12. El frontend muestra un mensaje de confirmación.
13. La mascota deja de aparecer en el catálogo general de mascotas disponibles.

Escenarios alternos:

- El usuario decide no confirmar la adopción y abandona la acción antes de enviar la petición.

Escenarios de error:

- **401 Unauthorized:** token faltante o inválido.
- **403 Forbidden:** un usuario intenta marcar como adoptada una publicación ajena.
- **404 Not Found:** la publicación no existe.
- **409 Conflict:** la mascota ya fue adoptada o no se encuentra disponible.

Postcondición: La mascota queda marcada como `ADOPTADO` y se retira del flujo general de exploración.

Endpoint utilizado:

`PUT /mascotas/{id}/adoptar`

Nota de alcance:

En esta versión, la confirmación de adopción cierra la publicación cambiando su estado a `ADOPTADO`. No se documenta como un sistema completo de contrato,

seguimiento legal o historial avanzado de adopciones.

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario dueño
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos

    Usuario->>UI: Accede a publication_view.html?id={id}
    UI-->>UI: Verifica token en sessionStorage
    UI-->>UI: Obtiene animalId desde la URL

    alt No existe token
        UI-->>Usuario: Redirige a login.html
    else Token presente
        UI->>API: GET /mascotas/detalle/{id}
        API->>DB: Busca publicación por animalId

        alt Publicación no encontrada
            DB-->>API: No existe publicación
            API-->>UI: HTTP 404 Not Found
            UI-->>Usuario: Muestra error de publicación no encontrada
        else Publicación encontrada
            DB-->>API: Retorna datos de mascota
            API-->>UI: HTTP 200 OK + datos de mascota
            UI-->>UI: Verifica si el usuario autenticado es dueño

            alt Usuario no es dueño
                UI-->>Usuario: No muestra opción "Confirmar adopción"
            else Usuario dueño
                UI-->>Usuario: Muestra botón "Confirmar adopción"

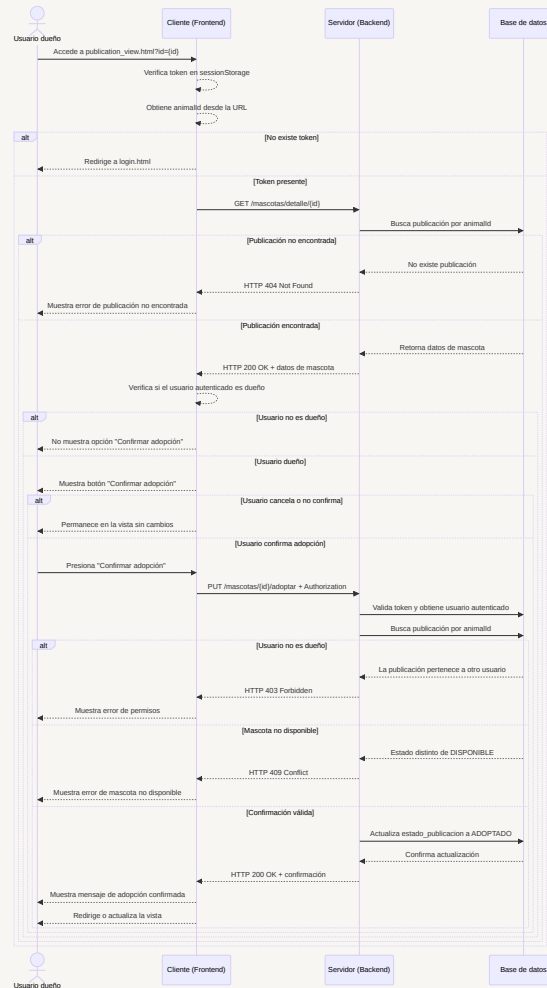
                alt Usuario cancela o no confirma
```

```

        UI-->>Usuario: Permanece en la vista si
n cambios
    else Usuario confirma adopción
        Usuario-->>UI: Presiona "Confirmar adop
ción"
        UI-->>API: PUT /mascotas/{id}/adoptar +
Authorization
        API-->>DB: Valida token y obtiene usuari
o autenticado
        API-->>DB: Busca publicación por animalI
d
        alt Usuario no es dueño
            DB-->>API: La publicación pertenece
a otro usuario
            API-->>UI: HTTP 403 Forbidden
            UI-->>Usuario: Muestra error de per
misos
        else Mascota no disponible
            DB-->>API: Estado distinto de DISPO
NIBLE
            API-->>UI: HTTP 409 Conflict
            UI-->>Usuario: Muestra error de mas
cota no disponible
        else Confirmación válida
            API-->>DB: Actualiza estado_publicac
ion a ADOPTADO
            DB-->>API: Confirma actualización
            API-->>UI: HTTP 200 OK + confirmaci
ón
            UI-->>Usuario: Muestra mensaje de a
dopción confirmada
            UI-->>Usuario: Redirige o actualiza
la vista
        end
    end
end

```

end
end



CU10: Panel de gestión de solicitudes de interés

Actores: Publicador de mascota.

Descripción:

Este caso de uso permite que el usuario que publicó una mascota consulte a las personas que han mostrado interés en adoptarla. El sistema presenta las mascotas publicadas por el usuario autenticado y, para cada una, la lista de interesados con su nombre, correo electrónico y fecha de interés.

Precondiciones:

- El usuario debe contar con una sesión activa.
- El token de sesión debe ser válido.

- El usuario puede tener o no mascotas publicadas.
- Pueden existir o no solicitudes de interés registradas sobre sus publicaciones.

Flujo principal:

1. El usuario publicador inicia sesión.
2. El usuario accede a la sección "Solicitudes".
3. El frontend verifica que exista una sesión activa.
4. El frontend realiza una petición `GET /solicitudes` con el token en `Authorization`.
5. El backend valida el token.
6. El backend identifica al usuario publicador.
7. El backend consulta las mascotas publicadas por el usuario autenticado.
8. El backend consulta los registros de interés asociados a dichas mascotas.
9. El backend obtiene nombre, correo electrónico y fecha de interés de cada usuario interesado.
10. El backend construye una respuesta agrupada por mascota.
11. El frontend recibe la respuesta y muestra cada mascota junto con sus solicitudes registradas.
12. El publicador puede visualizar los datos de contacto del interesado para iniciar comunicación.

Flujos alternativos:

- **A1. Usuario sin mascotas publicadas:** el backend devuelve una lista vacía y el frontend muestra un estado vacío.
- **A2. Mascota sin solicitudes:** el frontend muestra la mascota con un mensaje indicando que aún no tiene interesados.
- **A3. Mascota con varios interesados:** el frontend muestra la cantidad total de interesados y los datos de cada uno.

Escenarios de error:

- **401 Unauthorized:** el usuario intenta acceder sin sesión activa.
- **401 Unauthorized:** la petición no contiene token.

- **401 Unauthorized:** el token es inválido o la sesión expiró.

Postcondición: El publicador visualiza únicamente las solicitudes de interés correspondientes a sus propias mascotas. No se modifica información en la base de datos.

Endpoint utilizado:

GET /solicitudes

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Publicador de mascota
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos

    Usuario->>UI: Accede a solicitudes.html
    UI-->>UI: Verifica token en sessionStorage

    alt No existe token
        UI-->>Usuario: Redirige a login.html
    else Token presente
        UI->>API: GET /solicitudes + Authorization
        API->>DB: Valida token y obtiene usuario publicador

        alt Token inválido o expirado
            DB-->>API: No existe usuario asociado al token
            API-->>UI: HTTP 401 Unauthorized
            UI-->>Usuario: Redirige a login.html o muestra error de sesión
        else Token válido
            API->>DB: Consulta mascotas publicadas por el usuario
            DB-->>API: Retorna publicaciones del publicador

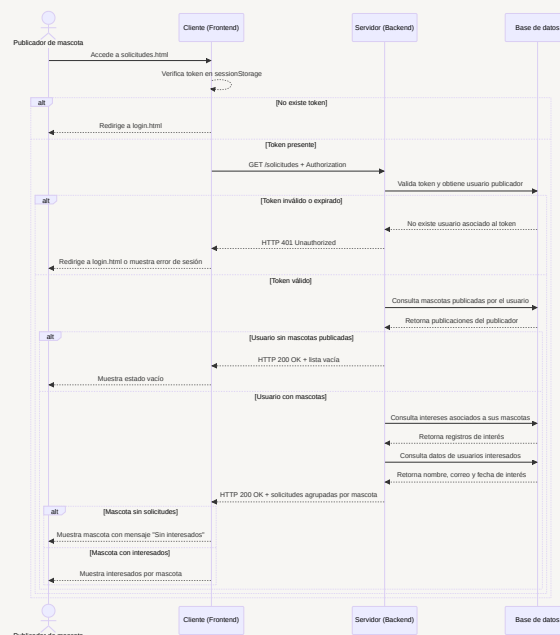
            alt Usuario sin mascotas publicadas
                API-->>UI: HTTP 200 OK + lista vacía
                UI-->>Usuario: Muestra estado vacío
            else Usuario con mascotas
```

```

s mascotas
    API->>DB: Consulta intereses asociados a su
    DB->>API: Retorna registros de interés
    API->>DB: Consulta datos de usuarios intere
    sados
    DB->>API: Retorna nombre, correo y fecha d
    e interés
    API->>UI: HTTP 200 OK + solicitudes agrupa
    das por mascota

alt Mascota sin solicitudes
    UI->>Usuario: Muestra mascota con mens
    aje "Sin interesados"
else Mascota con interesados
    UI->>Usuario: Muestra interesados por
    mascota
end
end
end
end
end

```



CU11: Eliminación de publicaciones de adopción

Actores: Publicador de mascota.

Descripción:

Este caso de uso permite que el usuario propietario de una publicación elimine una mascota publicada cuando esta ya no deba continuar disponible en la plataforma. La eliminación requiere una confirmación previa desde la interfaz y solo puede ser realizada por el dueño de la publicación.

Precondiciones:

- El usuario debe contar con sesión activa.
- El token debe ser válido.
- La publicación debe existir.
- El usuario autenticado debe ser propietario de la publicación.
- La publicación debe encontrarse en estado **DISPONIBLE**.

Flujo principal:

1. El usuario publicador inicia sesión.
2. El usuario accede al detalle o edición de una mascota propia.
3. El frontend identifica que el usuario autenticado es dueño de la publicación.
4. El frontend muestra el botón "Eliminar".
5. El usuario presiona "Eliminar".
6. El frontend muestra un Pop-In de confirmación.
7. El usuario confirma que desea eliminar la publicación.
8. El frontend envía **DELETE /mascotas/{id}** con el token en **Authorization**.
9. El backend valida el token.
10. El backend identifica al usuario autenticado.
11. El backend busca la publicación.
12. El backend verifica que la publicación pertenezca al usuario autenticado.
13. El backend verifica que la publicación esté **DISPONIBLE**.
14. El backend elimina la publicación de la base de datos.
15. El backend responde indicando que la publicación fue eliminada correctamente.
16. El frontend muestra un mensaje de confirmación.

17. El sistema redirige al usuario fuera del detalle de la publicación eliminada.

Flujos alternativos:

- **A1. Cancelación:** el usuario cancela en el Pop-In; el frontend cierra el modal y no envía la petición.
- **A2. Publicación con intereses asociados:** al eliminar la publicación, también deben eliminarse o dejar de mostrarse sus intereses asociados según la configuración de persistencia y consistencia de datos del backend.

Escenarios de error:

- **401 Unauthorized:** token faltante.
- **401 Unauthorized:** token inválido o sesión expirada.
- **403 Forbidden:** el usuario intenta eliminar una publicación ajena.
- **404 Not Found:** la publicación no existe.
- **409 Conflict:** la publicación ya no se encuentra disponible.

Postcondición: La publicación eliminada deja de aparecer en el catálogo general, en las publicaciones propias y en el panel de solicitudes.

Endpoint utilizado:

DELETE /mascotas/{id}

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Publicador de mascota
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos

    Usuario->>UI: Accede a update_mascot.html?id={id}
    UI-->>UI: Verifica token en sessionStorage
    UI-->>UI: Obtiene animalId desde la URL

    alt No existe token
        UI-->>Usuario: Redirige a login.html
    else Token presente
```

```

UI-->API: GET /mascotas/detalle/{id}
API-->DB: Busca publicación por animalId

alt Publicación no encontrada
    DB-->API: No existe publicación
    API-->UI: HTTP 404 Not Found
    UI-->Usuario: Muestra error de publicación no
encontrada
else Publicación encontrada
    DB-->API: Retorna datos de mascota
    API-->UI: HTTP 200 OK + datos de mascota
    UI-->UI: Verifica dueño y estado DISPONIBLE

alt Usuario no es dueño o publicación no dispon
ible
    UI-->Usuario: No muestra botón "Eliminar"
else Usuario dueño y publicación disponible
    UI-->Usuario: Muestra botón "Eliminar"
    Usuario-->UI: Presiona "Eliminar"
    UI-->Usuario: Muestra Pop-In de confirmaci
ón

alt Usuario cancela
    UI-->Usuario: Cierra Pop-In sin cambio
s
else Usuario confirma eliminación
    UI-->API: DELETE /mascotas/{id} + Autho
rization
    API-->DB: Valida token y obtiene usuari
o autenticado
    API-->DB: Busca publicación por animalI
d

alt Token inválido
    API-->UI: HTTP 401 Unauthorized
    UI-->Usuario: Muestra error de ses
ión

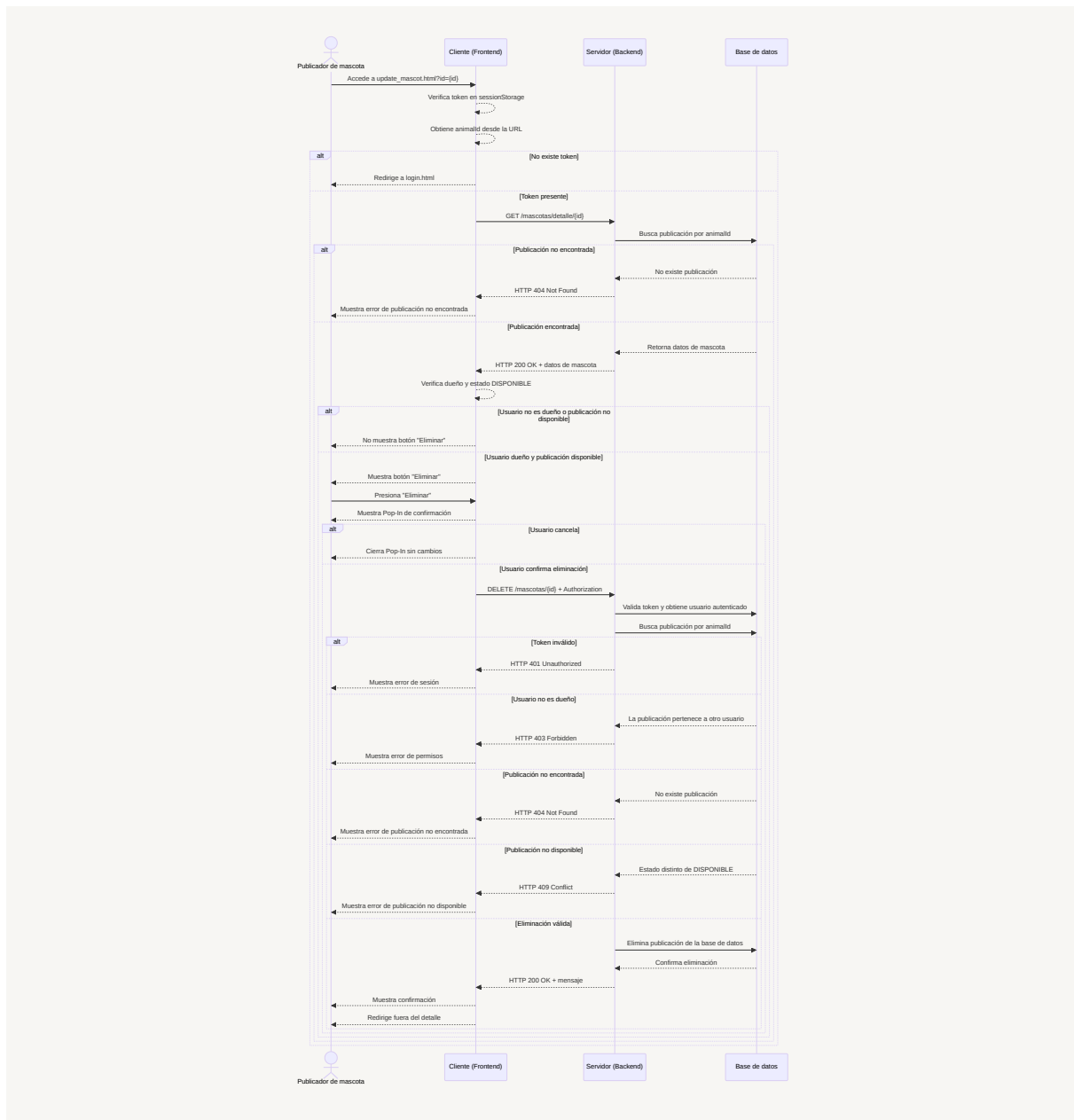
else Usuario no es dueño

```

```

a otro usuario
DB-->>API: La publicación pertenece
a otro usuario
API-->>UI: HTTP 403 Forbidden
UI-->>Usuario: Muestra error de per
misos
else Publicación no encontrada
DB-->>API: No existe publicación
API-->>UI: HTTP 404 Not Found
UI-->>Usuario: Muestra error de pub
licación no encontrada
else Publicación no disponible
DB-->>API: Estado distinto de DISPO
NIBLE
API-->>UI: HTTP 409 Conflict
UI-->>Usuario: Muestra error de pub
licación no disponible
else Eliminación válida
API->>DB: Elimina publicación de la
base de datos
DB-->>API: Confirma eliminación
API-->>UI: HTTP 200 OK + mensaje
UI-->>Usuario: Muestra confirmación
UI-->>Usuario: Redirige fuera del d
etalle
end
end
end
end
end
end
end

```



5. Requerimientos no funcionales

RNF1: Seguridad

Las contraseñas de los usuarios deben almacenarse utilizando un mecanismo de hash y no en texto plano. Las rutas protegidas deben validar el token recibido en el encabezado **Authorization**.

RNF2: Privacidad de la información

El sistema debe evitar la exposición pública de datos sensibles. Los datos de contacto de usuarios interesados solo se muestran al dueño autenticado de la

publicación correspondiente.

RNF3: Rendimiento

La carga de publicaciones y solicitudes debe realizarse en un tiempo razonable bajo condiciones normales de uso. El frontend debe mostrar estados de carga o mensajes informativos cuando no existan resultados.

RNF4: Usabilidad

La interfaz debe permitir que el usuario navegue por publicaciones, publique mascotas, marque interés, edite publicaciones propias, consulte solicitudes y elimine publicaciones de forma comprensible.

RNF5: Disponibilidad

La plataforma debe mantenerse disponible para permitir acceso a las funciones principales del sistema durante el periodo de operación de la entrega.

RNF6: Integridad de datos

El sistema debe evitar registros inconsistentes. Las operaciones de edición, adopción y eliminación deben validar propiedad, existencia de publicación y estado antes de modificar la base de datos.

6. Modelado de datos

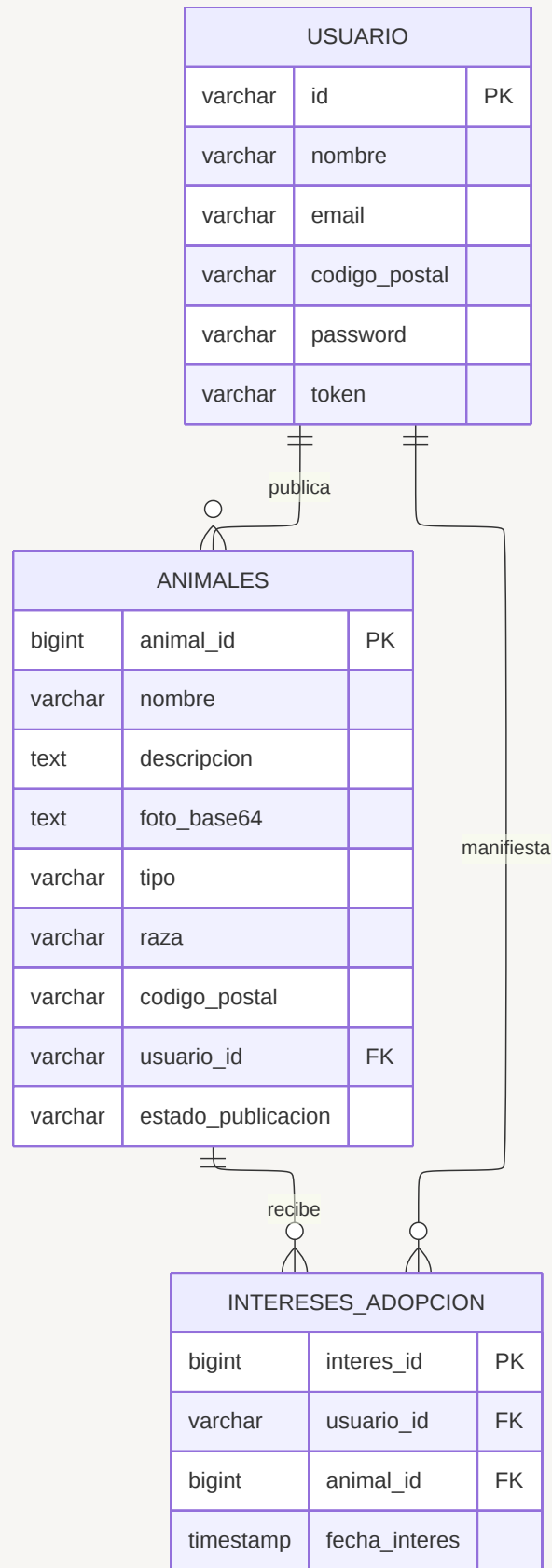
Modelo Entidad-Relación

```
erDiagram
    USUARIO ||--o{ ANIMALES : publica
    USUARIO ||--o{ INTERESES_ADOPCION : manifiesta
    ANIMALES ||--o{ INTERESES_ADOPCION : recibe

    USUARIO {
        varchar id PK
        varchar nombre
        varchar email
        varchar codigo_postal
        varchar password
        varchar token
    }
```

```
ANIMALES {  
    bigint animal_id PK  
    varchar nombre  
    text descripcion  
    text foto_base64  
    varchar tipo  
    varchar raza  
    varchar codigo_postal  
    varchar usuario_id FK  
    varchar estado_publicacion  
}
```

```
INTERESES_ADOPCION {  
    bigint interes_id PK  
    varchar usuario_id FK  
    bigint animal_id FK  
    timestamp fecha_interes  
}
```



Este modelo representa lo que sí maneja el sistema actual:

- Un **usuario** puede publicar muchas mascotas.
- Una **mascota** pertenece a un usuario publicador.
- Un **usuario** puede marcar interés en muchas mascotas.
- Una **mascota** puede recibir muchos intereses.
- **INTERESES_ADOPCION** funciona como tabla intermedia entre usuarios y animales.

Entidades principales

Usuario

Representa al actor principal del sistema. Permite registro, autenticación, publicación de mascotas y manifestación de interés.

Atributo	Tipo de dato	Propósito
id	VARCHAR	Identificador único del usuario.
nombre	VARCHAR	Nombre del usuario.
email	VARCHAR	Correo utilizado para login y contacto.
codigo_postal	VARCHAR	Ubicación aproximada del usuario.
password	VARCHAR	Contraseña hasheada.
token	VARCHAR	Token de sesión activo.

Animales

Representa las mascotas publicadas para adopción.

Atributo	Tipo de dato	Propósito
animal_id	BIGSERIAL	Identificador único de la mascota.
nombre	VARCHAR	Nombre de la mascota.
descripcion	TEXT	Descripción de la mascota.
foto_base64	TEXT	Fotografía codificada en Base64.
tipo	VARCHAR	Tipo o especie de mascota.
raza	VARCHAR	Raza o clasificación específica.
codigo_postal	VARCHAR	Ubicación aproximada.
usuario_id	VARCHAR	Identificador del usuario dueño de la publicación.

Atributo	Tipo de dato	Propósito
estado_publicacion	VARCHAR	Estado de la publicación, por ejemplo DISPONIBLE o ADOPTADO .

Intereses de adopción

Representa la relación entre usuarios interesados y mascotas publicadas.

Atributo	Tipo de dato	Propósito
interes_id	BIGSERIAL	Identificador único del interés.
usuario_id	VARCHAR	Usuario que manifestó interés.
animal_id	BIGINT	Mascota sobre la que se manifestó interés.
fecha_interes	TIMESTAMP	Fecha y hora en que se registró el interés.

7. Tecnologías utilizadas

7.1. Backend

- **Kotlin:** lenguaje principal del backend.
- **Spring Boot:** framework utilizado para exponer endpoints REST y organizar el proyecto.
- **Spring Data JPA:** acceso a base de datos mediante repositorios.
- **PostgreSQL:** sistema gestor de base de datos.
- **MessageDigest / SHA-256:** mecanismo utilizado para aplicar hash a contraseñas.

7.2. Frontend

- **HTML5:** estructura de las vistas.
- **CSS3:** estilos visuales del sistema.
- **JavaScript ES6:** lógica del cliente mediante módulos.
- **Fetch API:** comunicación HTTP con el backend.
- **sessionStorage:** almacenamiento temporal del token y datos básicos de sesión.

7.3. Complementos

- **GitHub:** control de versiones, ramas, Pull Requests y tags.
- **Notion:** documentación colaborativa y seguimiento del proyecto.
- **Postman / cURL:** pruebas de endpoints durante la integración.

8. Arquitectura general

8.1. Arquitectura Cliente-Servidor

El sistema adopta una arquitectura cliente-servidor. El frontend se ejecuta en el navegador y se encarga de presentar la interfaz, capturar eventos del usuario y realizar peticiones HTTP hacia el backend. El backend procesa las solicitudes, aplica reglas de negocio, consulta o modifica la base de datos y devuelve respuestas en formato JSON.

```

flowchart LR
    U[Usuario] --> N[Navegador Web]

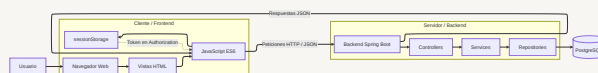
    subgraph Cliente [Cliente / Frontend]
        N --> HTML[Vistas HTML]
        HTML --> JS[JavaScript ES6]
        JS --> SS[sessionStorage]
    end

    JS -->|Peticiones HTTP / JSON| API[Backend Spring Boot]
    API -->|Respuestas JSON| JS

    subgraph Servidor [Servidor / Backend]
        API --> C[Controllers]
        C --> S[Services]
        S --> R[Repositories]
    end

    R --> DB[(PostgreSQL)]

    SS -. Token en Authorization .-> JS
  
```



8.2. Backend en capas

El backend mantiene una arquitectura por capas:

Controller → Service → Repository

- **Controller:** recibe peticiones HTTP y devuelve respuestas.
- **Service:** contiene la lógica de negocio, validaciones y reglas del sistema.
- **Repository:** se encarga del acceso a base de datos.
- **DTOs y Mappers:** separan los datos recibidos o enviados por la API del dominio interno y de las entidades de persistencia.

flowchart TD

Client[Frontend / Cliente]

subgraph Backend[Backend Spring Boot]

Controller[Controllers
MascotaController
SolicitudController
UsuarioController]

Service[Services
MascotaService
SolicitudService
UsuarioService]

Mapper[Mappers y DTOs
CreateMascotaRequest
MascotaResponse
SolicitudMascotaResponse]

Repository[Repositories
MascotaRepository
UsuarioRepository
InteresAdopcionRepository]

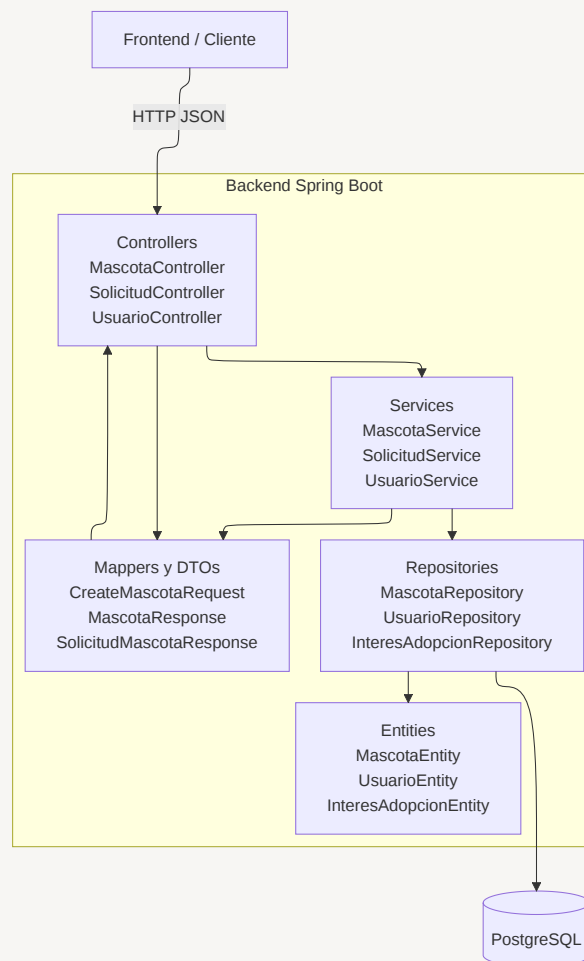
Entity[Entities
MascotaEntity
UsuarioEntity
InteresAdopcionEntity]

end

DB[(PostgreSQL)]

Client -->|HTTP JSON| Controller
 Controller --> Mapper
 Controller --> Service
 Service --> Repository
 Repository --> Entity
 Repository --> DB

Service --> Mapper
 Mapper --> Controller



8.3. Nuevos componentes de Iteración 4

En la Iteración 4 se agregan o consolidan componentes relacionados con:

- edición de publicaciones propias;
- confirmación de adopción;
- eliminación de publicaciones;

- consulta de solicitudes de interés mediante `SolicitudController`, `SolicitudService` y DTOs de solicitudes.

9. Flujo de autenticación

El sistema implementa autenticación mediante token asociado al usuario.

1. El usuario ingresa correo y contraseña en `login.html`.
2. El frontend envía `POST /usuarios/login`.
3. El backend valida credenciales.
4. El backend genera un token único.
5. El token se guarda asociado al usuario.
6. El frontend almacena el token en `sessionStorage`.
7. Para rutas protegidas, el frontend envía el token en el encabezado `Authorization`.
8. El backend valida el token contra la base de datos.
9. En logout, el backend invalida el token y el frontend limpia `sessionStorage`.

```
sequenceDiagram
```

```
    actor Usuario as Usuario
    participant UI as Cliente (Frontend)
    participant API as Servidor (Backend)
    participant DB as Base de datos
```

```
    Usuario->>UI: Ingresa correo y contraseña en login.html
    UI->>API: POST /usuarios/login
    API->>DB: Busca usuario por correo
```

```
    alt Credenciales inválidas
```

```
        DB-->>API: Usuario no encontrado o password incorre
```

```
cto
```

```
        API-->>UI: HTTP 401 Unauthorized
```

```
        UI-->>Usuario: Muestra error de credenciales
```

```
    else Credenciales válidas
```

```
        API->>API: Genera token único
```

```
        API->>DB: Guarda token asociado al usuario
```

```

    DB-->>API: Confirma actualización
    API-->>UI: HTTP 200 OK + token + userId
    UI-->>UI: Guarda token, email y userId en sessionStorage
end

    UI-->>Usuario: Redirige a home.html
end

Usuario-->>UI: Accede a vista protegida
UI-->>UI: Verifica token en sessionStorage

alt No hay token local
    UI-->>Usuario: Redirige a login.html
else Hay token local
    UI-->>API: Petición protegida + Authorization
    API-->>DB: Busca usuario por token

    alt Token inválido
        DB-->>API: No existe usuario asociado al token
        API-->>UI: HTTP 401 Unauthorized
        UI-->>UI: Limpia sessionStorage
        UI-->>Usuario: Redirige a login.html
    else Token válido
        DB-->>API: Retorna usuario autenticado
        API-->>UI: HTTP 200 OK + datos solicitados
        UI-->>Usuario: Muestra vista protegida
    end
end

Usuario-->>UI: Presiona Logout
UI-->>API: POST /usuarios/logout + Authorization
API-->>DB: Invalida token del usuario
DB-->>API: Confirma token eliminado
API-->>UI: HTTP 200 OK
UI-->>UI: Limpia sessionStorage
UI-->>Usuario: Redirige a login.html

```



10. Endpoints principales

Usuarios

Método	Endpoint	Descripción
POST	<code>/usuarios/register</code>	Registrar usuario.
POST	<code>/usuarios/login</code>	Iniciar sesión.
GET	<code>/usuarios/me</code>	Consultar usuario autenticado.
PUT	<code>/usuarios</code>	Actualizar datos de usuario.
POST	<code>/usuarios/logout</code>	Cerrar sesión.

Mascotas

Método	Endpoint	Descripción
POST	<code>/mascotas/publicar</code>	Publicar mascota.

Método	Endpoint	Descripción
GET	/mascotas	Consultar catálogo de mascotas.
GET	/mascotas?nombre={nombre}	Buscar mascotas por nombre.
GET	/mascotas?filtro=mis-publicaciones	Consultar publicaciones propias.
GET	/mascotas?filtro=me-interesa	Consultar mascotas marcadas con interés.
GET	/mascotas/detalle/{id}	Consultar detalle de mascota.
POST	/mascotas/{id}/interes	Registrar interés en una mascota.
PUT	/mascotas/editar/{id}	Editar publicación propia.
PUT	/mascotas/{id}/adoptar	Marcar mascota como adoptada.
DELETE	/mascotas/{id}	Eliminar publicación propia.

Solicitudes

Método	Endpoint	Descripción
GET	/solicitudes	Consultar solicitudes de interés recibidas por el publicador.

11. Limitaciones conocidas

- La visualización geoespacial mediante mapa por código postal queda como alcance futuro.
- La integración con The Dog API / The Cat API queda como alcance futuro.
- El envío automático de correos queda como alcance futuro; en esta versión se utiliza el panel de solicitudes para mostrar los datos de contacto al dueño de la publicación.
- La confirmación de adopción marca la publicación como **ADOPTADO**, pero no representa un flujo legal o administrativo completo de adopción.
- La eliminación de publicaciones se maneja desde el flujo del dueño autenticado; si se requiere historial de auditoría completo, en una versión futura convendría implementar borrado lógico.

12. Pruebas recomendadas para validación final

1. Registrar usuario nuevo.

2. Iniciar sesión.
3. Consultar perfil.
4. Actualizar datos de usuario.
5. Publicar una mascota.
6. Ver detalle de la mascota.
7. Buscar mascota por nombre.
8. Consultar "Mis publicaciones".
9. Registrar interés desde otro usuario.
10. Consultar el panel de solicitudes como dueño de la publicación.
11. Editar una publicación propia.
12. Intentar editar publicación ajena.
13. Confirmar adopción de una publicación propia.
14. Verificar que la mascota adoptada no aparezca en el catálogo general.
15. Intentar eliminar una publicación adoptada.
16. Eliminar una publicación disponible con Pop-In de confirmación.
17. Probar rutas protegidas sin token.
18. Probar token inválido.

13. Entregables de la Iteración 4

Backend

- Implementación de nuevos casos de uso.
- Persistencia en base de datos.
- Código funcional y estable.
- Tag requerido: 4.0.0.

Frontend

- Nuevas vistas funcionales.
- Integración completa con backend.

- Manejo de respuestas y errores.
- Tag requerido: 3.0.0 .

Documentación

- Documento actualizado a versión 4.0.
- Casos de uso implementados.
- Requerimientos funcionales actualizados.
- Arquitectura y flujo de autenticación revisados.
- Limitaciones conocidas actualizadas.